

MLFF campaign storage and I/O management specification

Status: Accepted current normative contract for the owner-driven storage subsystem. Historical STOR1-STOR5 design notes are archived in docs/history/mlff/STOR1_STOR5_HISTORICAL_DESIGN.md, and the retired DATA9B4 storage/restart contract is archived under docs/history/mlff/retired_specs/. Neither is current authority.

Purpose

This specification defines how the MLFF campaign subsystem manages persistent storage and I/O. Storage is strictly subordinate to the accepted P1-P7 scientific and currentness owners: it may change representation, retention, caching, deduplication, archival, recovery, admission, and I/O execution mechanics, and it may never change what the campaign means.

1. Non-negotiable protections

1. **Storage is never a second scientific authority.** It does not decide scientific identity, target membership, selected size, cross-validation acceptance, representative checkpoint choice, final publication membership, qualification outcome, locked activation, or a release verdict. It reads what the owners say.
2. **External inputs are indestructible.** User-supplied source datasets, training sets, replay trajectories, foundation models, and true-label reference material outside the campaign workspace are read-only regardless of path, symlink, configuration, or record reference.
3. **A reference is not an authority.** A path appearing in TOML, SQLite, JSON, a manifest, or an archive manifest confers no deletion authority. Every mutation additionally passes physical containment and ownership validation.
4. **Symlink targets are never traversed.** A campaign-owned symlink object may be unlinked; its target is never followed for traversal or deletion.
5. **Ambiguity retains.** Corrupt, unreadable, or unclassifiable owner state retains the affected artifacts until ownership is repaired or they are independently proven disposable.
6. **A report never authorizes a mutation.** Both report modes are read-only and advisory.
7. **Retired authority stays retired.** STOR1-STOR5 tier policy, workspace/runs conventions, active_process.json, PID/age/pathname rules, the retired evaluate/verify stages, DATA7/DATA8 stage folklore, and SELECT2 never regain destructive authority.

2. The one consequential flow

Every consequential storage mutation converges on a single path:

```
real P1-P7 owners
-> owner views
-> cross-owner inventory snapshot (transitive protection closure)
-> one canonical resolved storage policy
-> immutable owner-bound plan
-> explicit operator authorization
-> owner-local publication barrier + owner/currentness/filesystem revalidation
-> executor
-> durable audit
-> restart-equivalent product
```

There is exactly one destructive implementation. Reporting, planning, and execution are separate, and only execution mutates.

3. Retention is a cross-owner closure

Eligibility is computed over the **transitive dependency closure of every current or restartable owner**, not by asking each artifact's nominal owner in isolation.

Normative consequences:

- the current P7 publication pins the exact P5 published member checkpoint bytes for as long as that publication resolves and authenticates as current, **including after the P7 attempt retention reference has been released**;
- the current P4 terminal/selected authority pins the P3 immutable evidence its canonical loader and reconciliation chain require;
- a truthful `waiting_for_reference` pins every predecessor artifact needed to resume the exact frozen publication and qualification lineage, including the frozen external-reference request;
- protection is monotone: no owner's cache or history classification can override another current owner's requirement;
- historical evidence becomes archive- or reclamation-eligible only after no current or restartable descendant needs its hot representation;
- the closure is derived from current owner records on every invocation. There is no second persistent dependency database.

4. Race-safe mutation

A recent snapshot is not an authorization. P5 and P7 both publish an immutable object and then publish the current pointer that makes it current, so there is a legitimate window in which the object exists and nothing references it.

- Each owner exposes a **publication barrier** for one generation. The publisher holds it across object publication and pointer commit; any storage mutation that could touch that generation's evidence acquires the same barrier across revalidation and mutation.
- The storage-operation lease serializes storage operations against each other. It does **not** serialize storage against P1-P7 writers, and it is never treated as if it did.
- No broad CampaignStore transaction is held across hashing, compression, or recursive scans.
- Where an owner exposes no race-safe reclamation seam, the affected artifact is retained.
- P3's accepted publication-window evidence-graph fence remains valid substrate and is reused, not replaced.
- P7 durable objects/ are ordinary protected evidence and are never an orphan-collection target.

5. Actions and tiers

storage report

Owner views supply semantics; bounded physical metadata supplies size. Reports logical bytes, file and directory counts, owner and artifact class, current/historical/restart/cache/archive state, potential reclaim per action, unresolved owners, and protected inputs. The SHA-256 receipt cache is reported separately from CampaignStore authority.

storage report --deep

Explicit exact recursive physical accounting, symlink and ownership inspection, and largest-artifact detail. Read-only.

storage cleanup --tier safe

Zero scientific, restart, qualification, locked, and acceleration-cache capability loss. Candidates are only artifacts an owner has positively released: external record payloads no campaign state row references and that are outside the publication window; attempt-local bulk of a P7 attempt the owner has released, excluding the attempt record itself, whose terminality is monotonic; and abandoned storage-native staging with no open journal. Safe performs no acceleration-cache eviction.

storage cleanup --tier cache

Safe plus eviction of cache/index state whose owner can prove exact reconstruction *at that moment*. The normalized frame cache qualifies while the DATA2 source catalog resolves, the cache manifest binds that exact catalog digest, and every per-run source identity and control signature still matches; eviction then costs one source read per run and reproduces the identical authenticated cache. The SHA-256 receipt store is reclassified as reusable cache for accounting and ownership, and is retained by both tiers: it is the acceleration cache the running storage operation is itself writing to. Anything an owner cannot certify is retained, and a cache action over a campaign with no certified family is legitimately a no-op.

storage archive

A reversible authenticated cold representation of owner-declared historical reproducibility bulk. See section 7.

storage deduplicate

Owner-certified immutable deduplication. See section 8.

Retired tiers

recompute and compact are retired consequential-loss tiers. They are rejected by name; intentionally lossy history pruning requires a separate explicit product decision.

6. Resolved storage policy

One canonical resolved policy is shared by every CLI, config, and API entry point. It normalizes aliases before hashing, so equivalent spellings produce one identity, and it binds the requested action and tier, storage and scratch safety reserve, cache eviction limits, SQLite compaction thresholds, archive codec/level/expansion bounds, dedup realization and minimum file size, I/O worker limits, deep-audit bounds, lease timeout, and audit retention.

- Changing a material policy value invalidates an unapplied plan.
- A storage policy change never invalidates a P1-P7 scientific identity.
- Presentation-only options are deliberately outside the policy identity.
- Free bytes, inode headroom, and observed saturation are **execution observations** recorded on the plan, not policy. A changed disk causes admission revalidation, not scientific invalidation.
- No environment variable may widen deletion or archive authority; the resolver reads none.
- An unrecognized [storage] key is rejected rather than ignored.

7. Cold archive v2

Archive replaces hot bytes only when **all** of the following hold:

1. the owning artifact is historical or otherwise explicitly owner-declared cold-replaceable;
2. no current or restartable dependency closure requires its canonical hot representation;
3. no current owner resolver or currentness validator directly requires the canonical hot file;
4. explicit restore is sufficient to regain the promised historical capability;
5. the archive is authenticated and cataloged before any hot deletion.

Archive is **not** a transparent virtual filesystem. No P1-P7 loader is given an implicit “if missing, read the storage archive” fallback by this subsystem.

Locator containment

The archive catalog and its blobs live under one storage-owned authorized root. A manifest carries a canonical identity-owned relative locator, never an arbitrary filesystem path. An absolute locator, . . . , an empty or alias-normalizing component, a symlink escape, or a locator resolving outside the authorized root is rejected. A manifest field never authorizes reading an external file, even when those bytes satisfy the recorded digest. Member path safety is enforced separately and is not replaced by validating the outer locator. Restore from a user-supplied

external archive is not a supported feature of this package; adding it would require an explicit trust/import contract.

Bounded verification and extraction

Archive bytes are authenticated-but-untrusted until every check passes. Before and during extraction the subsystem enforces: supported schema and codec only; canonical workspace-relative member paths with no absolute path, `..`, empty component, normalization alias, or post-normalization duplicate; regular files and directories only, rejecting symlink, hard-link, device, FIFO, and socket members; manifest member-count and total-expanded-byte admission; a hard per-member size bound applied *while streaming*; a cumulative extracted-byte bound; a compressed-to-expanded ratio bound sufficient to refuse decompression amplification before extraction; a campaign-owned staging root with no traversal through archive-created symlinks; exact digest and size authentication before install; and no implicit overwrite of conflicting authoritative bytes.

Durable publication ordering

write/stage

- > flush + fsync content
- > atomic publish
- > persist the parent directory entry where supported
- > authenticate the published bytes
- > publish the dependent manifest/catalog/terminal receipt

Hot deletion follows an authenticated archive and a durable catalog, then a fresh owner/dependency revalidation, then removal of only still-authorized hot members, then a truthful terminal status. Restore is bounded authenticated staging, durable publication into canonical hot paths, final owner/content authentication, and only then a terminal receipt. The subsystem does not claim power-loss durability stronger than the filesystem provides; where a directory fsync is unavailable the content flush and atomic rename still hold.

The catalog is identity-keyed. There is no latest authority. Restoring bytes never promotes historical evidence to current.

8. Immutable deduplication

Eligibility is owner-certified immutability + exact content identity + owner-certified metadata compatibility + filesystem realization support + race-safe replacement.

- File type and required mode, ownership, and other owner-required metadata must match; equal bytes with divergent metadata are never hardlink aliases.
- A deduplicated family must have no accepted in-place content or material-metadata writer. Only superseded-generation roots participate, because every P1-P7 writer writes into the current generation.
- Replacement is a single atomic rename of a fully linked temporary, so an interruption can never leave a temporary path accepted as canonical.
- Cross-device or unsupported filesystems retain duplicate bytes without a correctness failure.
- Mutable SQLite state, active attempt scratch, and any owner-ambiguous file never participate.
- Dedup changes inode and ctime and therefore invalidates stat-keyed receipts. That is a cache miss and a revalidation, never a scientific state change.

9. Storage-native control plane

The subsystem owns durable state of its own under `.mdstats/storage/`: an identity-keyed archive catalog, archive manifests and blobs, restore journals, a bounded execution audit, operation-serialization state, and restore staging.

- The catalog, manifests, blobs, and journals required to locate, authenticate, resume, or restore a retained cold representation are never deleted or archived away by any action, including audit pruning.

- The execution audit is diagnostic evidence with bounded retention. Losing an old record cannot invalidate scientific currentness, and an incomplete operation is never recorded as complete.
- Operation-serialization state is operational liveness only. A crashed holder’s advisory lock is released by the kernel; recovery never infers authority from a PID, hostname, or pathname.
- No control-plane record carries a scientific currentness decision, and none can make a historical owner artifact current. Plans, audits, and manifests carry no secrets or machine credentials.

10. Admission

Storage does not introduce a second, weaker free-space floor. The campaign’s accepted `[execution].minimum_free_disk_gib` reserve and the `[storage]` safety reserve compose as the stricter of the two, because a reserve is a floor. Before a material storage operation the subsystem bounds free bytes against that reserve, inode headroom, and the operation’s **peak** temporary amplification — a staged archive blob before reclamation, a restore staging tree plus its installed copy, a dedup temporary link, a SQLite VACUUM rewriting the state database beside itself. An admission failure refuses the operation before any mutation and leaves the campaign exactly as it was. Storage pressure never changes target membership, precision, epochs, seed or qualification population, timestep, acceptance thresholds, or a locked policy. I/O concurrency is controlled independently of CPU worker count.

11. Interruption and terminality

Every multi-action operation has an explicit terminality contract.

- **Cleanup:** each removal is independently owner-authorized and race-safe. A crash after a subset is acceptable because each completed removal was individually safe. No terminal complete audit is published until every action in that execution reaches a verified terminal disposition. A retry re-inventories and re-plans rather than reusing the old remaining set. Rollback of already-safe deletions is not required.
- **Dedup:** each replacement is exact and idempotent; a retry reauthenticates both the canonical member and the content object before linking.
- **Archive:** archive bytes without an authenticated catalog never authorize hot deletion. An authenticated catalog may truthfully coexist with still-hot members after an interruption; the catalog records that state, and a retry reconciles which members remain and removes only those still authorized under a fresh owner-bound plan.
- **Restore:** partial staged or installed state is never accepted as complete. The terminal receipt follows final canonical-byte authentication. A retry is idempotent for already-present identical bytes and fails closed on conflicts.

12. Production qualification disposition

Routine implementation requires bounded functional, restart, and integrity tests plus representative storage/I/O measurement. Real campaign external-DFT qualification, long target-machine GPU production qualification, and environment-specific HPC storage qualification remain deferred and are not claimed by this specification.